Week 2: Querying and Optimizing the Urban Data Platform

During Week 1, you built the platform to store all datasets as Delta tables and provide an integrated dataset containing enriched taxi trips. Your task, this week, is to extend the platform by implementing reusable analytical queries, optimizing their performance, and building reusable analytical data products.

Task 1. Design Analytical Queries

Study the integrated dataset produced in Week 1 and design a collection of analytical queries for the following analyses.

1. Monthly taxi demand for each taxi zone.
2. Average trip distance under different weather conditions.
3. Relationship between air quality and taxi demand.
4. Taxi zones with the largest variation in demand under different weather conditions.
5. Peak travel hours for each day of the week.
6. Monthly trends in taxi demand.

Task 2. Implement the Analytical Queries

Implement all analytical queries using Spark SQL. The queries should execute directly on the integrated dataset and the underlying Delta tables produced in Week 1.

Task 3. Optimize Query Performance

Several analytical queries execute too slowly on the current platform. Optimize their performance. You may optimize queries that use either the integrated Delta table or the underlying Delta tables created in Week 1. At a minimum, investigate the effect of the following optimization techniques:

- Caching frequently accessed tables or intermediate results.
- Partition pruning by designing queries that read only the required partitions.
- Broadcast joins when joining the large Taxi Trips table with the small Weather, Air Quality, or Taxi Zone Lookup tables.
- Adaptive Query Execution (AQE) by comparing query performance with AQE enabled and disabled.

For each optimization:

- describe the optimization technique,
- explain why it is appropriate for the selected query,
- measure its impact on query execution time,

- verify that the optimized query produces the same results as the original implementation,
- discuss any trade-offs.

Support your conclusions by comparing query execution time before and after each optimization. Use EXPLAIN FORMATTED to identify changes in the physical execution plan (e.g., broadcast joins, partition pruning, or shuffle operations). You may also use the Spark UI to further analyze query execution.

Task 4. Build Reusable Data Products

Analysts do not want to execute complex SQL queries every time they need a report. Design and implement at least four reusable analytical data products that can be generated automatically from the integrated dataset.

Examples include:

- Daily Mobility Summary
- Taxi Zone Statistics
- Weather Impact Summary
- Air Quality Impact Summary
- Borough Mobility Summary

Each data product should:

- be generated automatically from the integrated dataset,
- be stored as a Delta table,
- include metadata such as the data source, creation time, refresh time, and schema version,
- support efficient querying.

For every data product, explain:

- who would use it,
- why it is useful,
- why it should be materialized instead of computed on demand.

Task 5. Evaluate the Platform

Evaluate the analytical performance of your platform by comparing the analytical queries before and after optimization. For each analytical query, measure:

- execution time,
- storage overhead of the analytical data products (where applicable),
- changes in the query execution plan (`EXPLAIN FORMATTED`),
- the effect of each optimization technique (e.g., caching, partition pruning, broadcast joins, AQE).

Verify that the optimized implementation produces the same results as the original implementation.

Based on your measurements, discuss:

- Which optimization produced the largest performance improvement?
- Which optimization had little or no effect? Why?
- Which queries remain computationally expensive?
- What characteristics of the data explain these results?
- If the municipality expanded the platform to process data from ten cities, what changes would you recommend?

Support all conclusions using experimental measurements rather than intuition.

Deliverables

Each group should submit

- Source code, i.e., the complete Spark project, including the analytical query library, reusable analytical data products, query optimization experiments, benchmarking code, and configuration files.
- A 3–5 page design report, containing the analytical requirements, analytical query design, analytical data products, optimization strategy, engineering decisions, and trade-offs.
- A short benchmark report, including the benchmark methodology, execution times before and after optimization, analysis of query execution plans (`EXPLAIN FORMATTED`), and discussion of performance.
- A README describing how to run the analytical queries, generate the analytical data products, and reproduce the benchmark experiments.